

# Data Collection

## Data Collection and Data Sources

### Overview

This project skips outside data sources altogether - rather, it builds its own made-up stream of logistics events that mirror cargo flows through Iraq. Its aim is to craft a repeatable flow of location-tagged records updated fast enough for live tracking, pattern spotting without labels, training models with examples, or predicting daily operations.

All created data flows straight into AWS DynamoDB, where it's saved and later pulled from; this database runs the show for analysis, visuals, or training smart systems. Everything works through code. This way, each phase starting from making events up to saving and grabbing them back, can be repeated exactly when needed.

### Data Source Information

All simulation info comes from a Python event engine (`server.py`), then lands in an AWS DynamoDB table named Shipments - this holds everything live. Instead of just saving records, it keeps trucks, vessels, paths, total miles, forecast features, and all other active simulation entities; at the same time, it streams updates to a WebSocket-powered front end and feeds traits into ML guesses like KMeans groups delay-risk predictions. It runs on DynamoDB inside eu-north-1, sorting entries by truckId up top. Several types of synthetic data are produced by the simulation layer, including truck movement events created by stepping through OSRM-generated routes, ship movement events based on random GPS jitter, full polyline route geometries obtained through the OSRM API, cumulative distance profiles computed using the haversine formula, and machine learning features such as delay probabilities generated by pretrained sklearn models. The only external data dependency is the Open Source Routing Machine (OSRM) API, which provides realistic drivable routes in Iraq by returning full road polylines, true route lengths, and geometry needed for accurate ETA estimation.

## Data Collection Methods

The project does not scrape or fetch static datasets. Instead, it implements a real-time streaming data generator that emits events every second.

## Event Simulation Pipeline

When the user clicks Start Simulation, the backend:

1. Reads user parameters
  - Number of trucks
  - Origin hub
  - Destination hub
  - Departure time
2. Computes optimal route Calls OSRM to obtain:
  - Polyline coordinates
  - Total route length
  - Estimated travel duration
3. Initializes all trucks Each truck is saved to DynamoDB with:
  - truckId
  - start, end
  - lat, lon
  - route (full polyline, serialized for DynamoDB)
  - cumdist (cumulative distance array)
  - total\_km
  - idx (route pointer)
  - ETA, status, etc.
4. Starts the streaming simulation loop Every second:
  - Trucks advance along the route polyline
  - Ships drift randomly

- ML inference is computed (cluster + delay risk)
- All updates are written to DynamoDB

Updated state is broadcast to the frontend via WebSocket (/ws)

## Tools Used

- FastAPI — backend server + WebSocket streaming
- boto3 — DynamoDB read/write operations
- requests — OSRM routing calls
- asyncio — high-frequency simulation loop
- joblib — loading ML models

All data collection logic exists in server.py, ensuring full reproducibility.

## Data Structure and Format

Every record stored in DynamoDB follows a consistent JSON-serializable schema. Example truck record:

```
{ "truckId": "TRUCK-1", "start": "Basra Port", "end": "Baghdad Depot", "lat": 31.2044,
  "lon": 46.8091, "route": [ {"lat": 30.5081, "lon": 47.7835}, {"lat": 30.61, "lon": 47.12}, ...
], "cumdist": [0, 1.2, 2.8, ...], "total_km": 545.3, "idx": 42, "eta": "2025-12-10T19:50:00Z",
  "status": "IN_TRANSIT", "plan_speed_kmh": 78.2, "cluster": 1, "delayRisk": 0.32 }
```

Data Formats used:

| Component           | Format                                      |
|---------------------|---------------------------------------------|
| Truck and ship data | JSON records stored in DynamoDB             |
| Route geometry      | GeoJSON from OSRM, stored as list of points |
| WebSocket packets   | JSON                                        |
| ML models           | .pkl (joblib)                               |

## Linking to Data and Reproducibility

Although DynamoDB is not publicly accessible, the following resources ensure full transparency and reproducibility:

### **Code Repositories+Backend Simulation Server (FastAPI + DynamoDB):**

[https://github.com/sazandkhalid/supply\\_chain\\_tracker](https://github.com/sazandkhalid/supply_chain_tracker)

### **Frontend Visualization (Leaflet + WebSocket):**

<https://sazdsan.georgetown.domains>

## Reproducible Data Generation

Check README.md

## Transparency and Replicability

The entire pipeline, from event generation to machine learning inference, is fully open, deterministic, and well-documented, ensuring complete transparency throughout the system. The use of synthetic yet realistic simulation logic, deterministic routing from OSRM, traceable event updates, reproducible ML inference via static .pkl models, and publicly accessible backend, frontend, and ML code allows every step of the workflow to be replicated without ambiguity. This means that anyone can reproduce the simulation logic, rebuild the DynamoDB dataset, rerun the clustering and delay risk models, and recreate the live geospatial visualizations exactly as demonstrated. Together, these elements satisfy all requirements for reproducibility and open technical methodology.